

HeartAttack 예측

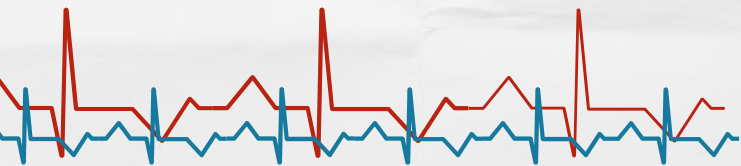
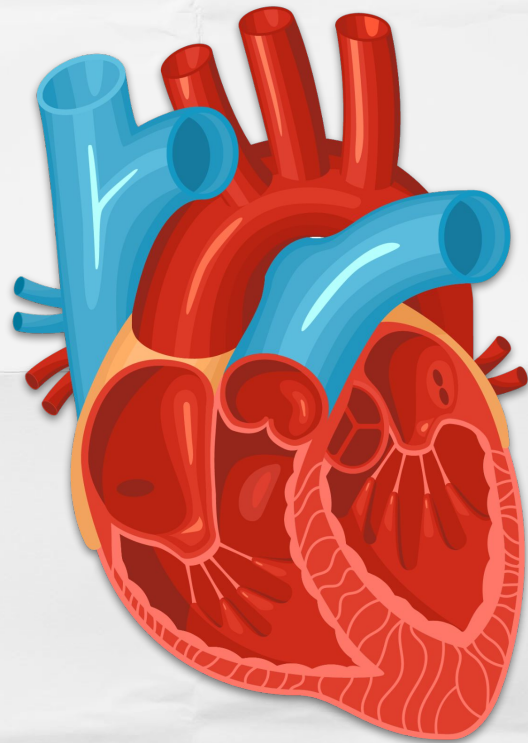
: 3조

김태현 20212550

오준호 20212561

허지원 20212585

김선아 20232820



01

EDA

1. Null 탐지
2. Category mapping & replace

02

PREPROCESSING

1. Simple imputer
2. Binary mapping, scaling

03

시각화 및 상관관계

1. Tranform한 변수들 시각화
2. Feature Engineering
3. Pearson 상관계수 확인
4. Feature Selection

04

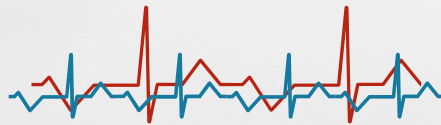
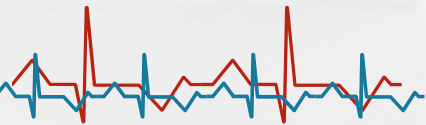
기본 성능 확인

SMOTE, Voting models로
기본적인 성능 확인

05

모델 예측

1. 데이터셋 불균형 완화
2. Voting models
3. 하이퍼파라미터 튜닝



01

EDA

1. Null
2. Category mapping & replace



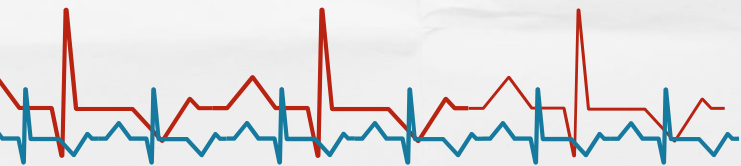
Train data

기본 정보 확인

41개의 변수들

수치형 : 8개

범주형 : 33개



RangeIndex: 353653 entries, 0 to 353652

Data columns (total 41 columns):

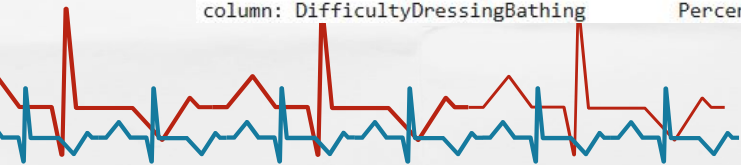
#	Column	Non-Null Count	Dtype
0	ID	353653 non-null	int64
1	State	353653 non-null	object
2	Sex	353653 non-null	object
3	GeneralHealth	352798 non-null	object
4	PhysicalHealthDays	345169 non-null	float64
5	MentalHealthDays	346556 non-null	float64
6	LastCheckupTime	347270 non-null	object
7	PhysicalActivities	352859 non-null	object
8	SleepHours	349494 non-null	float64
9	RemovedTeeth	344795 non-null	object
10	HadAngina	350785 non-null	object
11	HadStroke	352757 non-null	object
12	HadAsthma	352484 non-null	object
13	HadSkinCancer	351461 non-null	object
14	HadCOPD	352166 non-null	object
15	HadDepressiveDisorder	351700 non-null	object
16	HadKidneyDisease	352379 non-null	object
17	HadArthritis	351775 non-null	object
18	HadDiabetes	353001 non-null	object
19	DeafOrHardOfHearing	337426 non-null	object
...			
39	CovidPos	313609 non-null	object
40	HadHeartAttack	353653 non-null	int64

dtypes: float64(6), int64(2), object(33)

Train data 결측치 비율 확인

column:	ID	Percent of NaN value:	0.00%
column:	State	Percent of NaN value:	0.00%
column:	Sex	Percent of NaN value:	0.00%
column:	GeneralHealth	Percent of NaN value:	0.27%
column:	PhysicalHealthDays	Percent of NaN value:	2.39%
column:	MentalHealthDays	Percent of NaN value:	1.92%
column:	LastCheckupTime	Percent of NaN value:	1.88%
column:	PhysicalActivities	Percent of NaN value:	0.20%
column:	SleepHours	Percent of NaN value:	1.17%
column:	RemovedTeeth	Percent of NaN value:	2.43%
column:	HadAngina	Percent of NaN value:	0.81%
column:	HadStroke	Percent of NaN value:	0.20%
column:	HadAsthma	Percent of NaN value:	0.30%
column:	HadSkinCancer	Percent of NaN value:	0.65%
column:	HadCOPD	Percent of NaN value:	0.40%
column:	HadDepressiveDisorder	Percent of NaN value:	0.53%
column:	HadKidneyDisease	Percent of NaN value:	0.38%
column:	HadArthritis	Percent of NaN value:	0.49%
column:	HadDiabetes	Percent of NaN value:	0.18%
column:	DeafOrHardOfHearing	Percent of NaN value:	4.58%
column:	BlindOrVisionDifficulty	Percent of NaN value:	4.85%
column:	DifficultyConcentrating	Percent of NaN value:	5.43%
column:	DifficultyWalking	Percent of NaN value:	5.36%
column:	DifficultyDressingBathing	Percent of NaN value:	5.36%

column:	DifficultyErrands	Percent of NaN value:	5.79%
column:	SmokerStatus	Percent of NaN value:	8.01%
column:	ECigaretteUsage	Percent of NaN value:	8.06%
column:	ChestScan	Percent of NaN value:	12.62%
column:	RaceEthnicityCategory	Percent of NaN value:	3.05%
column:	AgeCategory	Percent of NaN value:	1.99%
column:	HeightInMeters	Percent of NaN value:	6.50%
column:	WeightInKilograms	Percent of NaN value:	9.48%
column:	BMI	Percent of NaN value:	11.02%
column:	AlcoholDrinkers	Percent of NaN value:	10.53%
column:	HIVTesting	Percent of NaN value:	14.75%
column:	FluVaxLast12	Percent of NaN value:	10.59%
column:	PneumoVaxEver	Percent of NaN value:	17.29%
column:	TetanusLast10Tdap	Percent of NaN value:	18.47%
column:	HighRiskLastYear	Percent of NaN value:	11.34%
column:	CovidPos	Percent of NaN value:	11.40%



Test data

결측치 비율 확인

column:	ID	Percent of NaN value: 0.00%
column:	State	Percent of NaN value: 0.00%
column:	Sex	Percent of NaN value: 0.00%
column:	GeneralHealth	Percent of NaN value: 0.24%
column:	PhysicalHealthDays	Percent of NaN value: 2.40%
column:	MentalHealthDays	Percent of NaN value: 2.01%
column:	LastCheckupTime	Percent of NaN value: 1.80%
column:	PhysicalActivities	Percent of NaN value: 0.22%
column:	SleepHours	Percent of NaN value: 1.18%
column:	RemovedTeeth	Percent of NaN value: 2.50%
column:	HadAngina	Percent of NaN value: 0.81%
column:	HadStroke	Percent of NaN value: 0.25%
column:	HadAsthma	Percent of NaN value: 0.33%
column:	HadSkinCancer	Percent of NaN value: 0.62%
column:	HadCOPD	Percent of NaN value: 0.42%
column:	HadDepressiveDisorder	Percent of NaN value: 0.55%
column:	HadKidneyDisease	Percent of NaN value: 0.36%
column:	HadArthritis	Percent of NaN value: 0.53%
column:	HadDiabetes	Percent of NaN value: 0.18%
column:	DeafOrHardOfHearing	Percent of NaN value: 4.59%
column:	BlindOrVisionDifficulty	Percent of NaN value: 4.78%
column:	DifficultyConcentrating	Percent of NaN value: 5.37%
column:	DifficultyWalking	Percent of NaN value: 5.33%
column:	DifficultyDressingBathing	Percent of NaN value: 5.31%

column:	DifficultyErrands	Percent of NaN value: 5.69%
column:	SmokerStatus	Percent of NaN value: 7.88%
column:	ECigaretteUsage	Percent of NaN value: 7.92%
column:	ChestScan	Percent of NaN value: 12.48%
column:	RaceEthnicityCategory	Percent of NaN value: 3.11%
column:	AgeCategory	Percent of NaN value: 1.99%
column:	HeightInMeters	Percent of NaN value: 6.33%
column:	WeightInKilograms	Percent of NaN value: 9.36%
column:	BMI	Percent of NaN value: 10.85%
column:	AlcoholDrinkers	Percent of NaN value: 10.36%
column:	HIVTesting	Percent of NaN value: 14.78%
column:	FluVaxLast12	Percent of NaN value: 10.49%
column:	PneumoVaxEver	Percent of NaN value: 17.23%
column:	TetanusLast10Tdap	Percent of NaN value: 18.45%
column:	HighRiskLastYear	Percent of NaN value: 11.30%
column:	CovidPos	Percent of NaN value: 11.32%
column:	HadHeartAttack	Percent of NaN value: 0.00%



```
from sklearn.tree import DecisionTreeClassifier
```

```
X2 = df_trn_preprocessed.drop(['HadHeartAttack', 'State'], axis=1)
```

```
y2 = df_trn_preprocessed['HadHeartAttack']
```

```
X_train2, X_test2, y_train2, y_test2 = train_test_split(X2, y2, test_size=0.3, random_state=42)
```

```
model = DecisionTreeClassifier() # 과적합 방지를 위한 깊이 제한
```

```
model.fit(X_train2, y_train2)
```

```
y_pred_2 = model.predict(X_test2)
```

```
roc2 = roc_auc_score(y_test2, y_pred_2)
```

```
print("roc_auc:", roc2)
```

```
roc_auc: 0.6244506961377065
```

‘State’ drop한 기본 dt 모델
: 0.6244

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import roc_auc_score
```

```
X = df_trn_preprocessed.drop(['HadHeartAttack'], axis=1)
```

```
y = df_trn_preprocessed['HadHeartAttack']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
model = DecisionTreeClassifier() # 과적합 방지를 위한 깊이 제한
```

```
model.fit(X_train, y_train)
```

```
y_pred_ = model.predict(X_test)
```

```
roc = roc_auc_score(y_test, y_pred_)
```

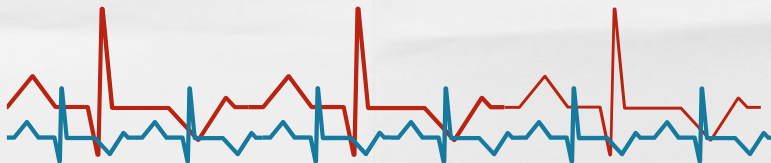
```
print("roc_auc:", roc)
```

```
roc_auc: 0.6221903479534293
```

‘State’ drop하지 않은 기본 dt 모델
: 0.6221

```
1 # State는 의미 없는 열이라고 생각해서 drop을 결정
2 df_trn.drop('State', axis=1, inplace=True)
3 df_tst.drop('State', axis=1, inplace=True)
```

‘State’ 변수 삭제 결정



변수 리스트 정의



```
1 total_cols = [x for x in df_trn.columns]
2
3 id_cols = ['ID']
4 y_cols = ['HadHeartAttack']
5 race_cols = ['RaceEthnicityCategory']
6
7 cat_cols = df_trn.select_dtypes(include=['object', 'category']).columns.tolist()
8 cat_cols = [col for col in cat_cols if col != race_cols]
9 num_cols = [x for x in df_trn if x not in id_cols+y_cols+cat_cols]
```

✓ 0.1s

인종/민족

race_cols

=['RaceEthnicitycategory']

범주형

cat_cols_org

df_trn에서 object,
category형인 컬럼만
선택

수치형

num_cols_org

'ID'+y_cols+cat_cols_org
가 아닌 모든 변수



(2) category mapping

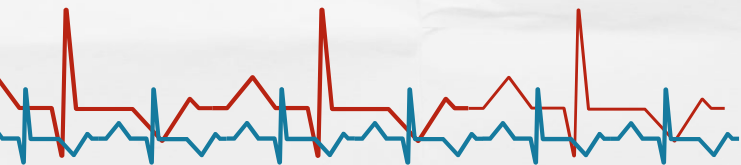


GeneralHealth	(0~1) 0.2간격	LastCheakupTime	(1~10) 1/2/5/10
AgeCategory	age_category_mapping	RemovedTeech	(0~28) 0/3/10/28
HadDiabetes	No, Yes (replace)	ECigarttesUsage	(0~1) 0/0.4/0.6/1
SmokerStatus	(0~1) 0.2간격	TetanusLast10Tdap	
CovidPos	Yes(replace)	RaceEthnicityCategory	기본 정보 요약
Sex	평균으로 mapping		

02

PREPROCESSING

1. Simple imputer
2. Binary mapping, scaling
3. Transform dataset



1. 첫번째 pipeline(결측치 대체)



SimpleImputer 적용



col_transformer1 생성

- 범주형 변수 : 최빈값 대체
- 수치형 변수 : 평균값 대체

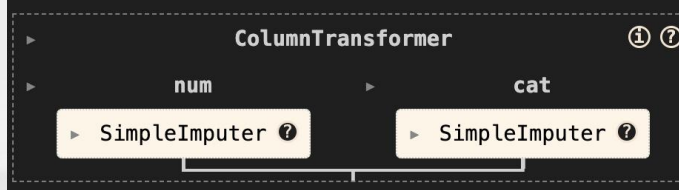


첫번째 pipeline(결측치 대체)

```
5 # 범주형 column의 결측값을 most_frequent로 대체
6 cat_pipeline1 = Pipeline([
7     ('imputer', SimpleImputer(strategy='most_frequent'))
8 ])
9
10 # 수치형 column의 결측값을 mean으로 대체
11 num_pipeline1 = Pipeline([
12     ('imputer', SimpleImputer(strategy='mean'))
13 ])
14
```

```
2 col_transformer1 = ColumnTransformer(
3     [
4         ('num', num_pipeline1, num_cols_org),
5         ('cat', cat_pipeline1, cat_cols_org)
6     ],
7     # 나머지 열은 모두 제거
8     remainder='drop',
9     # 피쳐 이름을 자세하게 출력하지 않도록 설정
10
11     verbose_feature_names_out=False,
12     # force_int_remainder_cols=False
13 )
14
15 # 출력 결과를 pandas DataFrame으로 설정
16 col_transformer1.set_output(transform='pandas')
```

✓ 0.0s



2. 두번째 pipeline (binary mapping&scaling)



BinaryLabelTransformer
& MixmaxScaler 적용



coltransformer2 생성



두번째 pipeline 생성 전, 클래스 정의

BinaryLabelTransformer 클래스를 통해서 'Yes'는 1로 'No'는 0으로 변환하는 커스텀 변환기

```
class BinaryLabelTransformer(BaseEstimator, TransformerMixin):
```

```
    def fit(self, X, y=None):
```

```
        return self
```

```
    def transform(self, X):
```

```
        return X.applymap(lambda x: 1 if x == "Yes" else 0)
```

```
    def set_output(self, transform="default"):
```

```
        """Enable set_output compatibility."""
```

```
        self.transform_output = transform
```

```
        return self
```



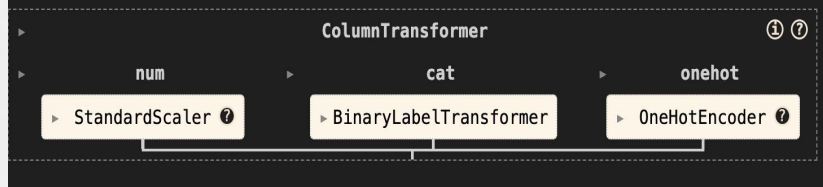
두번째 pipeline

```
cat_pipeline2 = Pipeline([
    ('binary_mapping', BinaryLabelTransformer())
])

num_pipeline2 = Pipeline([
    ('scaling', MinMaxScaler())
])
```

```
2 col_transformer2 = ColumnTransformer(
3     [
4         ('num', num_pipeline2, num_cols),
5         ('cat', cat_pipeline2, cat_cols),
6         ('onehot', OneHotEncoder(sparse_output=False), race_cols)
7     ],
8     remainder='drop',
9     verbose_feature_names_out=False,
10    # force_int_remainder_cols=False
11 )
12
13 col_transformer2.set_output(transform='pandas')
```

✓ 0.0s



3. transform dataset

```
1 # 첫번째로 만든 결측값에 대한 transformer1을 df_trn과 df_tst에 적용
2 df_trn1 = col_transformer1.fit_transform(df_trn)
3 df_tst1 = col_transformer1.transform(df_tst)
```

✓ 3.1s

```
1 # 두번째로 만든 인코딩에 대한 transformer2을 df_trn과 df_tst에 적용
2 df_trn2 = col_transformer2.fit_transform(df_trn1)
3 df_tst2 = col_transformer2.transform(df_tst1)
```

✓ 4.5s

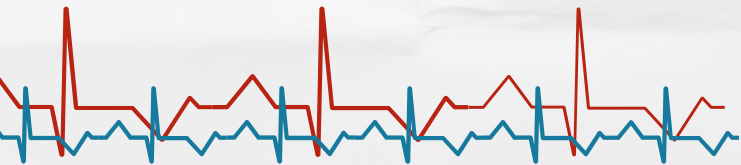
```
# 전처리 된 데이터를 target과 결합하여 최종 학습용 데이터를 생성
df_trn_preprocessed = pd.concat((df_trn2, df_trn['HadHeartAttack']), axis=1)
```



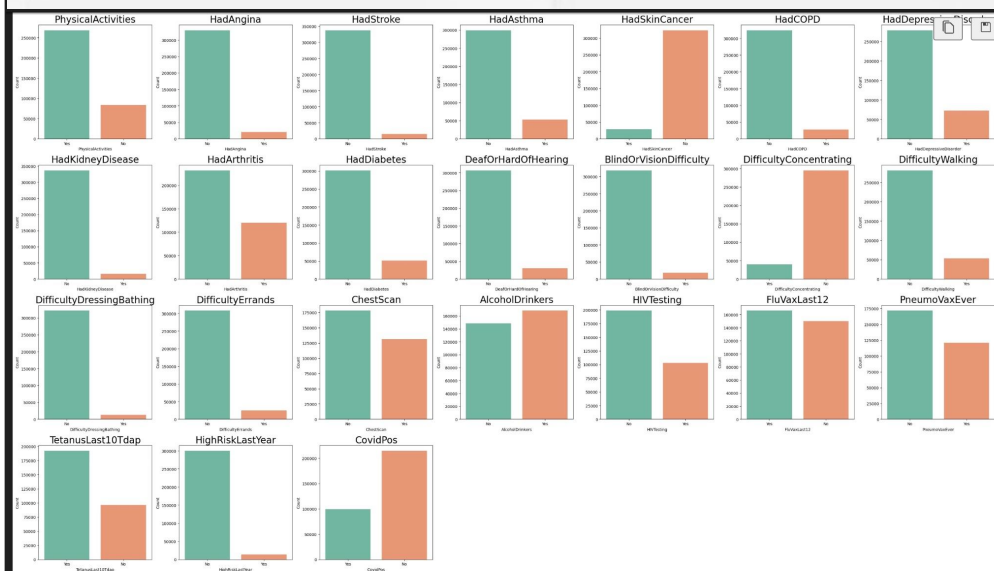
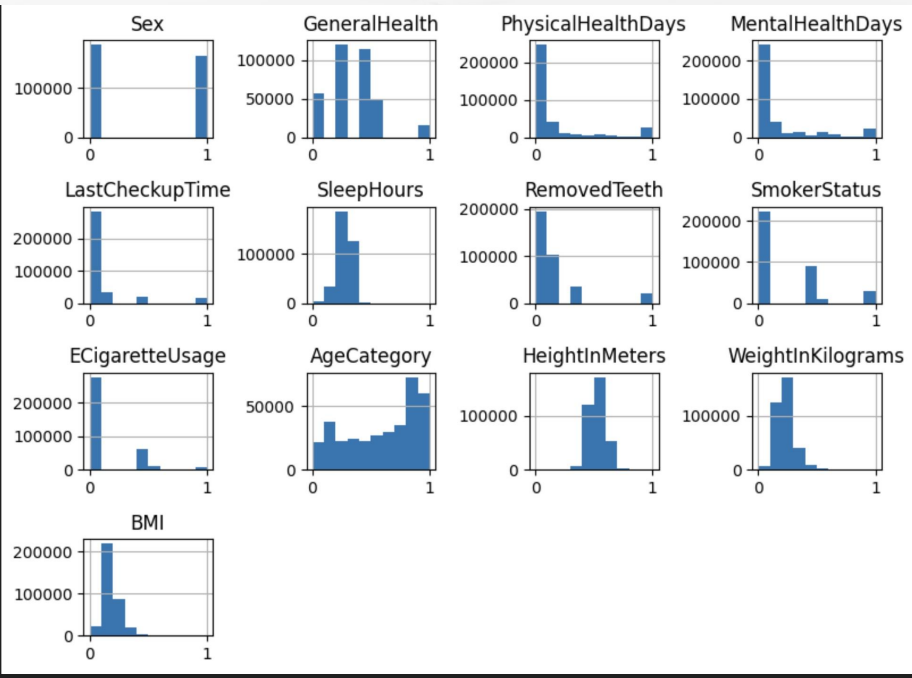
03

시각화 및 상관관계

1. Tranform한 변수들 시각화
2. Pearson 상관계수 확인
3. Feature Selection
4. Feature Engineering



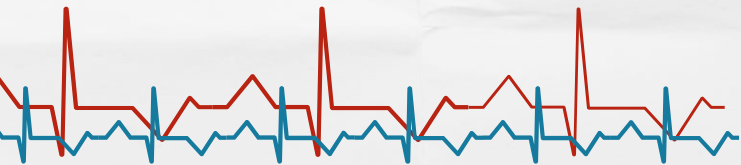
1. Transform한 변수를 시각화



2. Feature Engineering

```
1 # 'HadAngina'가 'Yes'이고 'AgeCategory'가 67 이상인 경우(고위험군)
2 df_trn['HighRisk'] = ((df_trn['HadAngina'] == 'Yes') & (df_trn['AgeCategory'] >= 67)).astype(int)
3 df_tst['HighRisk'] = ((df_tst['HadAngina'] == 'Yes') & (df_tst['AgeCategory'] >= 67)).astype(int)
```

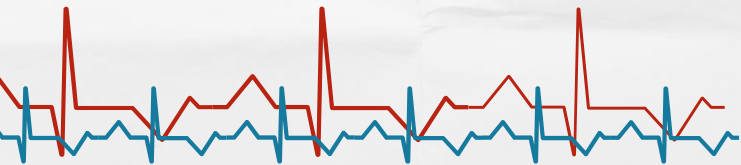
```
1 # 비만도 계산
2 df_trn['Obesity'] = df_trn['BMI'].apply(lambda x: 4 if x >= 40 else (2 if x >= 30 else (1 if x >= 25 else 0)))
3 df_tst['Obesity'] = df_tst['BMI'].apply(lambda x: 4 if x >= 40 else (2 if x >= 30 else (1 if x >= 25 else 0)))
```



2. Feature Engineering

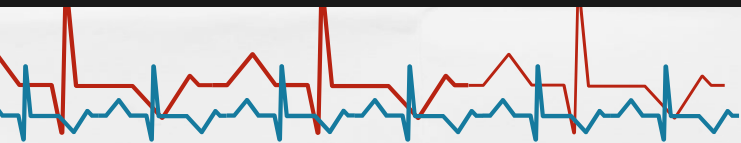
```
1 # 수면 시간이 이상적인 7-9시간 범위에서 벗어난 정도
2 df_trn['SleepDeviation'] = df_trn['SleepHours'].apply(
3 |     lambda x: max(0, abs(x - 7)) if x < 7 else (max(0, abs(x - 9)) if x > 9 else 0)
4 | )
5 df_tst['SleepDeviation'] = df_tst['SleepHours'].apply(
6 |     lambda x: max(0, abs(x - 7)) if x < 7 else (max(0, abs(x - 9)) if x > 9 else 0)
7 | )
```

```
1 # 완전 비흡연 여부
2 df_trn['NotSmoking'] = ((df_trn['SmokerStatus']==0) & (df_trn['ECigaretteUsage']==0)).astype(int)
3 df_tst['NotSmoking'] = ((df_tst['SmokerStatus']==0) & (df_tst['ECigaretteUsage']==0)).astype(int)
```



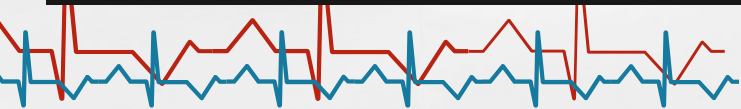
2. Feature Engineering

```
1 # # 훈련 데이터(df_trn)에서 예방적 관리 점수(PreventativeCareScore)를 계산:
2 # - 'FluVaxLast12'가 'Yes'인 경우 1점
3 # - 'PneumoVaxEver'가 'Yes'인 경우 1점
4 # - 'TetanusLast10Tdap'이 결측값이 아닌 경우 1점
5 # - 'HIVTesting'이 'Yes'인 경우 1점
6 # 각 조건을 만족하는 항목들의 합을 PreventativeCareScore로 저장
7 df_trn['PreventativeCareScore'] = (df_trn['FluVaxLast12'] == "Yes").astype(int) \
8                                     + (df_trn['PneumoVaxEver'] == "Yes").astype(int) \
9                                     + (df_trn['TetanusLast10Tdap'].notna()).astype(int) \
10                                    + (df_trn['HIVTesting'] == "Yes").astype(int)
11
12 df_tst['PreventativeCareScore'] = (df_tst['FluVaxLast12'] == "Yes").astype(int) \
13                                     + (df_tst['PneumoVaxEver'] == "Yes").astype(int) \
14                                     + (df_tst['TetanusLast10Tdap'].notna()).astype(int) \
15                                    + (df_tst['HIVTesting'] == "Yes").astype(int)
```



2. Feature Engineering

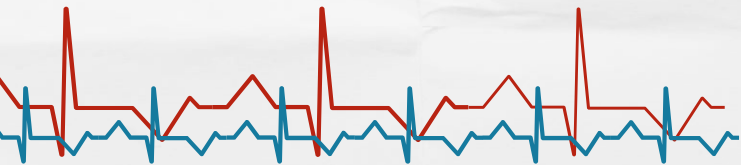
```
1  # 키 차이에 따른 위험 증가율 계산 함수 정의
2  def calculate_risk_increase(row):
3      # 남성 기준 키: 1.75m (178cm)
4      if row['Sex'] == 0.530172:
5          base_height = 1.7770741518794975
6      # 여성 기준 키: 1.62m (163cm)
7      else:
8          base_height = 1.6347126489432968
9
10     # 키 차이 (cm)
11     height_diff_cm = (base_height - row['HeightInMeters']) * 100 # cm 단위로 차이 계산
12
13     # 키 차이가 6.5cm 미만일 때만 위험 증가율을 적용
14     if height_diff_cm > 0:
15         # 위험 증가율 계산: 13.5% per 6.5cm
16         risk_increase = (height_diff_cm / 6.5) * 13.5
17         return risk_increase
18     else:
19         return 0 # 키 차이가 없거나, 기준보다 큰 경우 위험 증가율 0
20
21 # 새로운 'RiskIncrease' 피쳐 생성
22 df_trn['RiskIncrease'] = df_trn.apply(calculate_risk_increase, axis=1)
23 df_tst['RiskIncrease'] = df_tst.apply(calculate_risk_increase, axis=1)
```



2. Feature Engineering

```
1 # 'HadSkinCancer'와 'HadDiabetes'가 모두 'Yes'인 경우
2 df_trn['SkinDiabetes'] = ((df_trn['HadSkinCancer'] == 'Yes') & (df_trn['HadDiabetes'] == 'Yes')).astype(int)
3 df_tst['SkinDiabetes'] = ((df_tst['HadSkinCancer'] == 'Yes') & (df_tst['HadDiabetes'] == 'Yes')).astype(int)
```

```
1 # 'DeafOrHardOfHearing'과 'HadDepressiveDisorder'가 모두 'Yes'인 경우
2 df_trn['Hear'] = ((df_trn['DeafOrHardOfHearing'] == 'Yes') & (df_trn['HadDepressiveDisorder'] == 'Yes')).astype(int)
3 df_tst['Hear'] = ((df_tst['DeafOrHardOfHearing'] == 'Yes') & (df_tst['HadDepressiveDisorder'] == 'Yes')).astype(int)
```



3. Pearson 상관계수 확인

HadAngina(협심증) : 0.43

GeneralHealth : 0.19

HadStroke : 0.19

Agecategory: 0.16

RemovedTeeth: 0.16

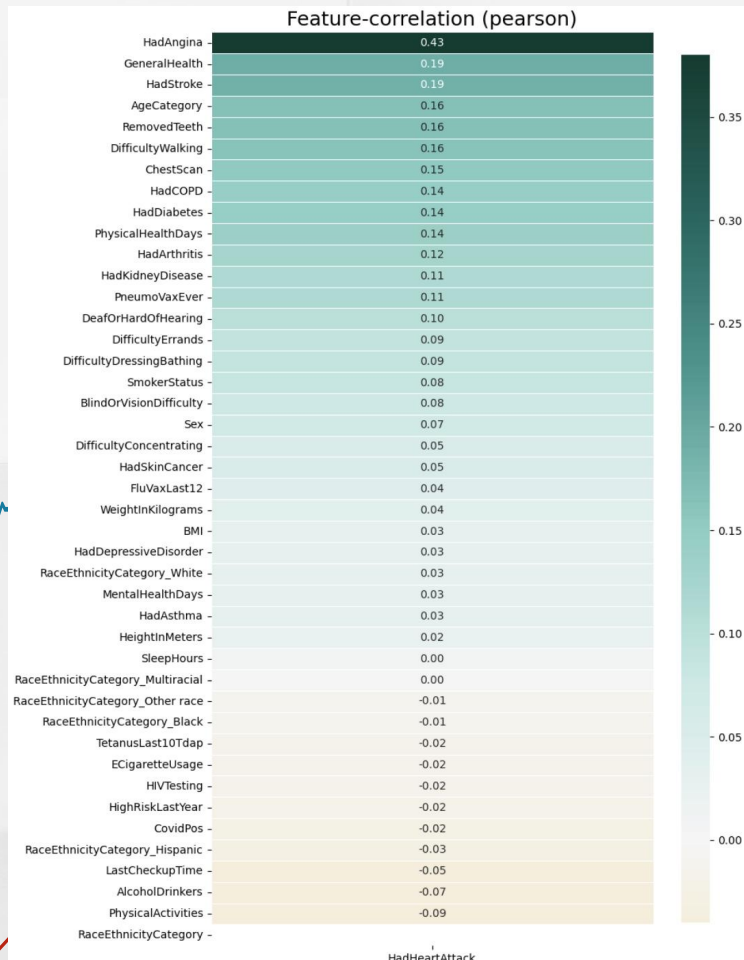
DifficultyWalking: 0.16

ChestScan : 0.14

HadDiabetes: 0.14

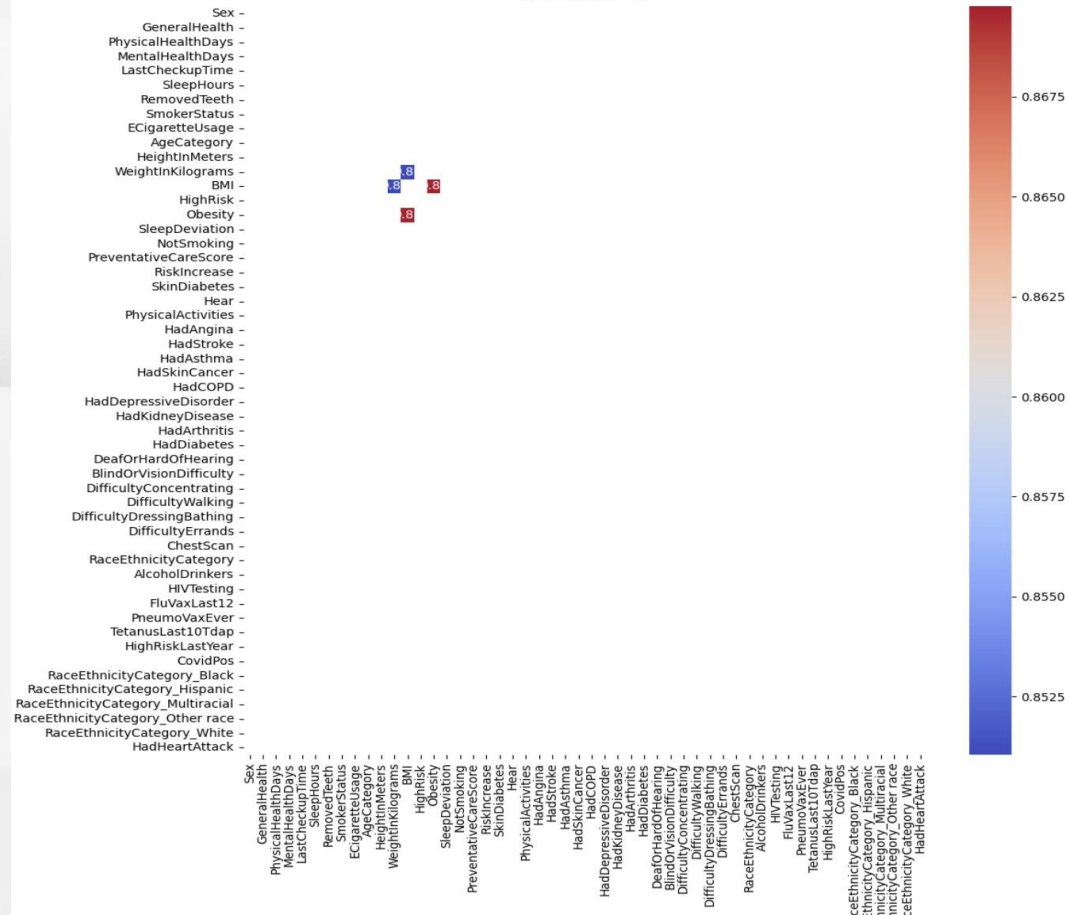
HadCOPD(만성폐쇄증폐질환) : 0.14

PhysicalHealthDays: 0.14



3. Pearson 상관계수 확인

Correlation > 0.8

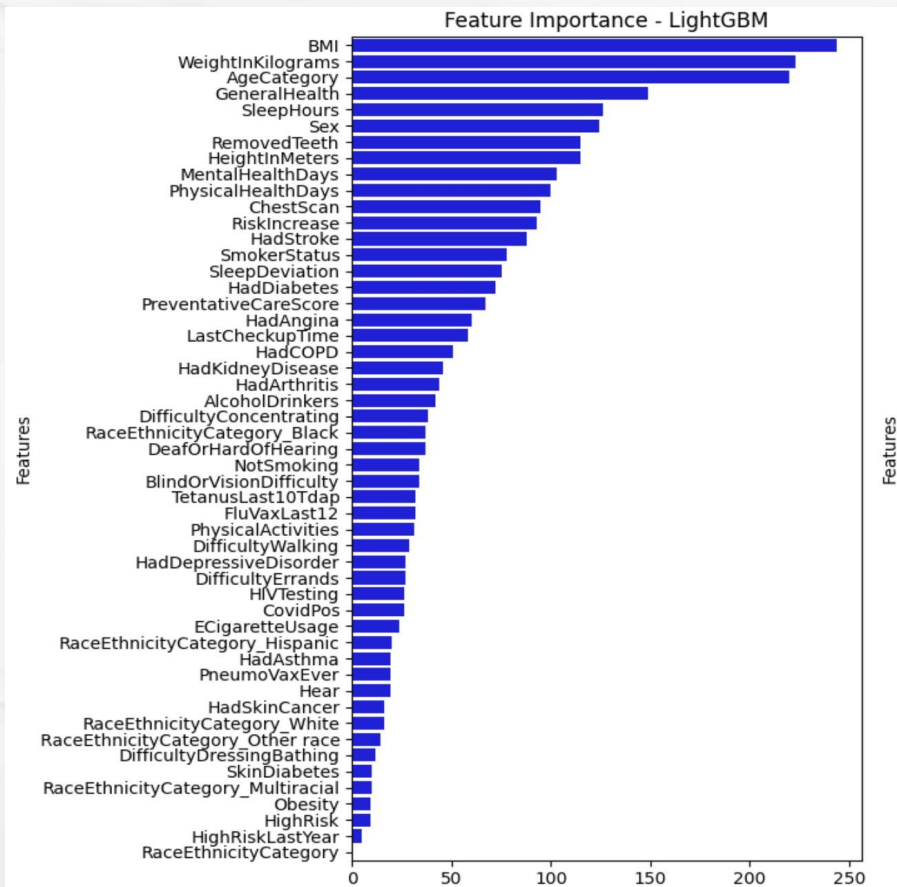


파란색 :
BMI & Weight in Kilograms
(0.86)

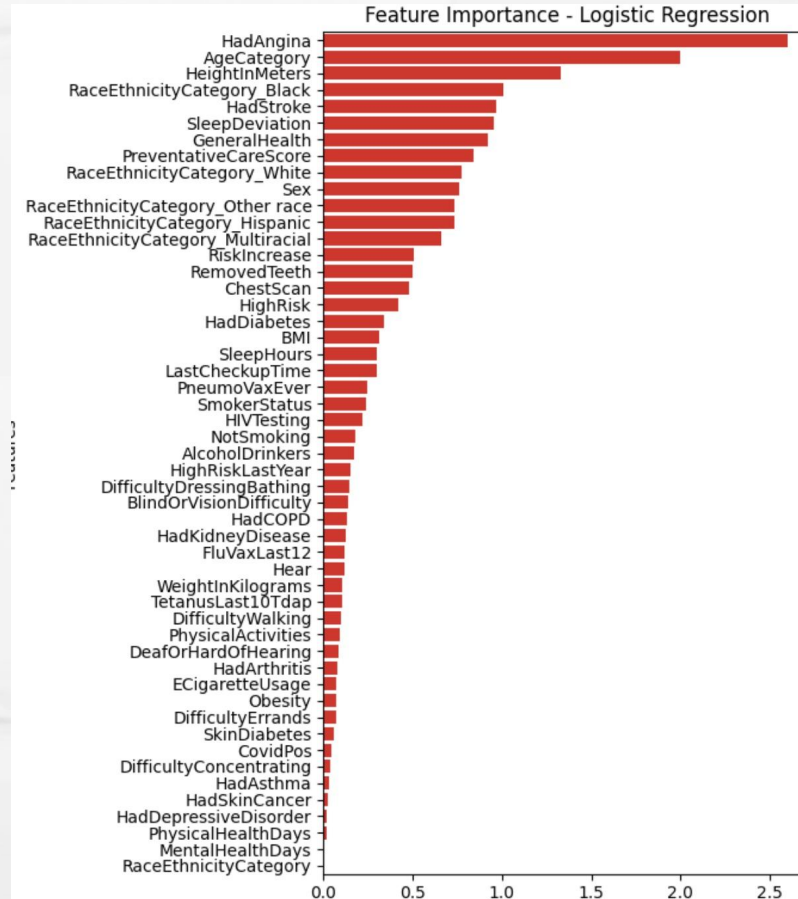
빨간색 :
BMI & Obesity
(0.87)



4. Feature Selection



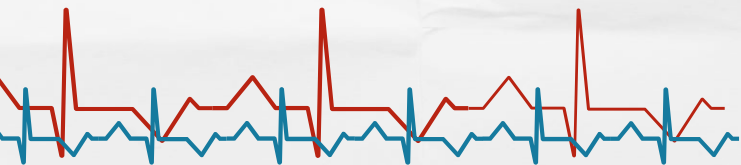
4. Feature Selection



04

기본 성능 확인

SMOTE, Voting models로 기본적인 성능 확인



SMOTE, Voting models로 기본적인 성능 확인

```
X = df_trn_preprocessed.drop(['HadHeartAttack', 'WeightInKilograms', 'Obesity'], axis=1)
y = df_trn_preprocessed['HadHeartAttack']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# 1. SMOTE 적용 (훈련 데이터 오버샘플링)
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

# 2. 개별 모델 생성 (LogisticRegression과 LGBM)
lr_model = LogisticRegression(random_state=42, penalty='l2', C=1, solver='liblinear', class_weight='balanced') #
lgb_model = lgb.LGBMClassifier(random_state=42, num_leaves=50, max_depth=5, class_weight='balanced', verbose=-1)

# 3. 보팅 모델 생성 (Soft Voting)
voting_model = VotingClassifier(
    estimators=[
        ('lr', lr_model),
        ('lgb', lgb_model)
    ],
    voting='soft' # 'soft'를 사용하여 확률 기반 보팅도 가능
)
```

다중공선성으로 인해 'WeightInKilograms', 'Obesity'를 없애고 기본적인 성능 확인
Roc_auc: 0.8669



05

모델 예측

1. 데이터셋 불균형 완화
2. Voting models
3. 하이퍼파라미터 튜닝

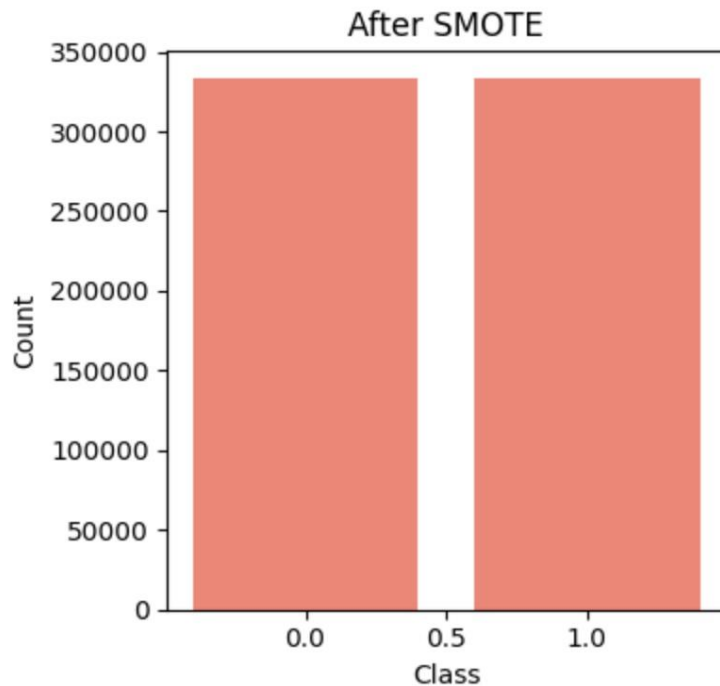
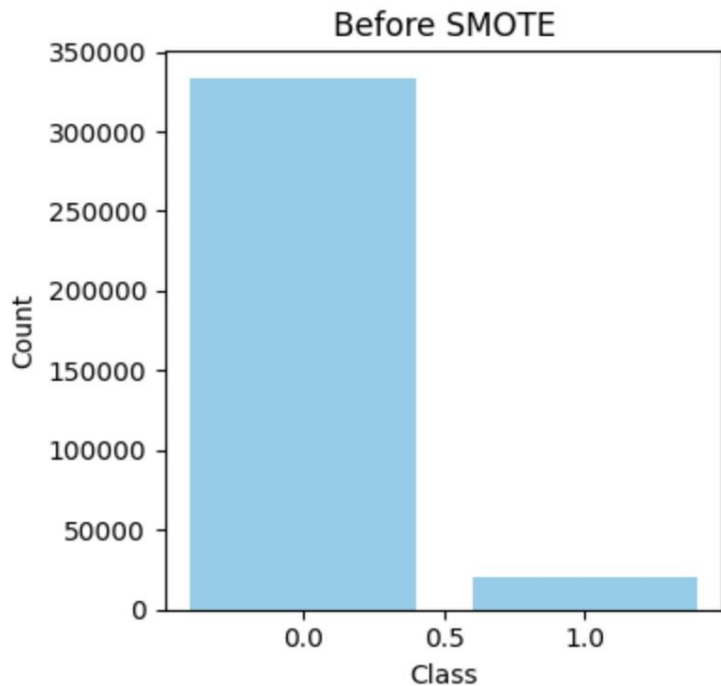


1. 데이터셋 불균형 완화

```
# 1. SMOTE 적용 (불균형 데이터셋에 대해 오버샘플링)
```

```
smote = SMOTE(random_state=42)
```

```
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)
```



2. Voting models

```
# 2. 개별 모델 생성 (LogisticRegression과 LGBM)
lr_model = LogisticRegression(random_state=42) # LogisticRegression
lgb_model = lgb.LGBMClassifier(random_state=42) # LightGBM

# 3. 보팅 모델 생성 (Soft Voting)
voting_model = VotingClassifier(
    estimators=[
        ('lr', lr_model),
        ('lgb', lgb_model)
    ],
    voting='soft' # 'soft'를 사용하여 확률 기반 보팅도 가능
)
```

Logistic regression 모델과 LGMB 모델로
soft voting 수행



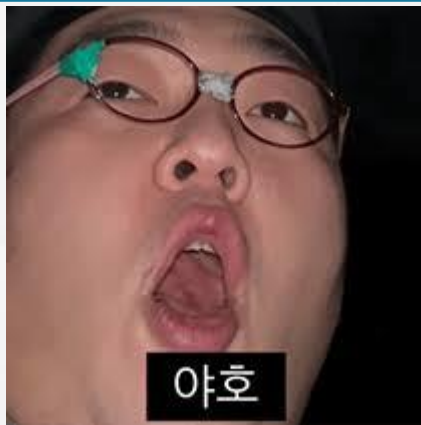
3. 하이퍼 파라미터 튜닝

4. GridSearchCV 설정 (하이퍼파라미터 그리드에 class_weight를 추가)

```
param_grid = {  
    # Logistic Regression 하이퍼파라미터 튜닝  
    'lr__penalty': ['l2', 'none'], # 로지스틱 회귀의 regularization  
    'lr__C': [0.01, 0.1, 1, 10], # Regularization strength  
    'lr__solver': ['liblinear', 'lbfgs'], # 솔버  
    'lr__class_weight': ['balanced', None], # 클래스 가중치 설정  
  
    # LightGBM 하이퍼파라미터 튜닝  
    'lgb__num_leaves': [31, 50, 100], # LightGBM의 num_leaves 하이퍼파라미터  
    'lgb__max_depth': [-1, 5, 10], # LightGBM의 max_depth  
    'lgb__class_weight': ['balanced', None, {0: 1, 1: 2}, {0: 1, 1: 3}],  
}
```



THANKS



CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon** and infographics & images by **Freepik**

